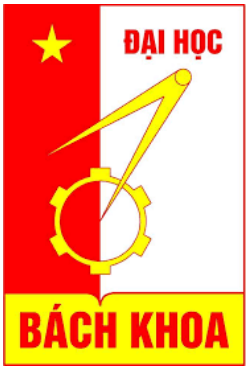
ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN & TRUYỀN THÔNG



BÁO CÁO

BÀI 3: TẠO BẢNG KÝ HIỆU

Tên môn học: Xây dựng chương trình dịch **Mã môn học:** IT3323

Tên GVHD: TS. Nguyễn Thị Thu Hương **Mã lớp:** 171159

Sinh viên thực hiện: Đinh Văn Phạm Việt **MSSV:** 20235870

Hà Nội, ngày 06 tháng 11 năm 2026

MỤC LỤC

CHƯƠNG 1. Kết quả thực hiện chương trình 1

1.1 Biên dịch và chạy chương trình 1

1.2 Màn hình kết quả 1

CHƯƠNG 2. Mã nguồn các hàm cần hoàn thiện 2

2.1 Hàm `makeCharType` 2

2.2 Hàm `makeArrayType` 2

2.3 Hàm `createConstantObject` 3

2.4 Hàm `createTypeObject` 3

2.5 Hàm `createFunctionObject` 4

2.6 Hàm `createParameterObject` 4

2.7 Hai điều chỉnh bổ sung 5

DANH MỤC HÌNH VẼ

Hình 1.1 Kết quả xây dựng và in bảng ký hiệu của chương trình PRG. 1

CHƯƠNG 1. Kết quả thực hiện chương trình

1.1 Biên dịch và chạy chương trình

Thiết lập môi trường: Cần cài đặt GCC và Make, đồng thời thêm thư mục chứa các công cụ này vào biến PATH để PowerShell nhận được lệnh `gcc` và `make`.

Biên dịch và chạy chương trình: Mở PowerShell tại thư mục mã nguồn và thực hiện:

```
make
make run
```

1.2 Màn hình kết quả

```
PS C:\Users\ADMIN\Desktop\KPL\Symtab1_incompleted> make run
.\symtab.exe
Program PRG
    Const c1 = 10
    Const c2 = 'a'
    Type t1 = Arr(10,Int)
    Var v1 : Int
    Var v2 : Arr(10,Arr(10,Int))
    Function f : Int
        Param p1 : Int
        Param VAR p2 : Char

    Procedure p
        Param v1 : Int
        Const c1 = 'a'
        Const c3 = 10
        Type t1 = Int
        Type t2 = Arr(10,Int)
        Var v2 : Arr(10,Int)
        Var v3 : Char

PS C:\Users\ADMIN\Desktop\KPL\Symtab1_incompleted>
```

Hình 1.1: Kết quả xây dựng và in bảng ký hiệu của chương trình PRG.

CHƯƠNG 2. Mã nguồn các hàm cần hoàn thiện

2.1 Hàm `makeCharType`

Tạo một đối tượng kiểu và gán lớp kiểu là `TP_CHAR`.

```
Type* makeCharType(void) {
//TODO
  Type* type = (Type*) malloc(sizeof(Type));
  type->typeClass = TP_CHAR;
  return type;
}
```

2.2 Hàm `makeArrayType`

Tạo kiểu mảng, lưu số phần tử và con trỏ đến kiểu phần tử; kiểu phần tử có thể tiếp tục là một kiểu mảng.

```
Type* makeArrayType(int arraySize, Type* elementType) {
  //TODO
  Type* type = (Type*) malloc(sizeof(Type));
  type->typeClass = TP_ARRAY;
  type->arraySize = arraySize;
  type->elementType = elementType;
  return type;
}
```

2.3 Hàm `createConstantObject`

Tạo đối tượng hằng, lưu tên, loại đối tượng và cấp phát vùng thuộc tính chứa giá trị hằng.

```
Object* createConstantObject(char *name) {
  //TODO
  Object* obj = (Object*) malloc(sizeof(Object));
  strcpy(obj->name, name);
  obj->kind = OBJ_CONSTANT;
  obj->constAttrs =
      (ConstantAttributes*) malloc(sizeof(ConstantAttributes));
  obj->constAttrs->value = NULL;
  return obj;
}
```

2.4 Hàm `createTypeObject`

Tạo đối tượng kiểu tự định nghĩa và cấp phát vùng thuộc tính lưu kiểu thực tế.

```
Object* createTypeObject(char *name) {
  //TODO
  Object* obj = (Object*) malloc(sizeof(Object));
  strcpy(obj->name, name);
  obj->kind = OBJ_TYPE;
  obj->typeAttrs =
      (TypeAttributes*) malloc(sizeof(TypeAttributes));
  obj->typeAttrs->actualType = NULL;
  return obj;
}
```

2.5 Hàm `createFunctionObject`

Tạo đối tượng hàm, khởi tạo danh sách tham số và tạo phạm vi riêng có phạm vi ngoài là phạm vi hiện tại.

```
Object* createFunctionObject(char *name) {
  //TODO
  Object* obj = (Object*) malloc(sizeof(Object));
  strcpy(obj->name, name);
  obj->kind = OBJ_FUNCTION;
  obj->funcAttrs =
      (FunctionAttributes*) malloc(sizeof(FunctionAttributes));
  obj->funcAttrs->paramList = NULL;
  obj->funcAttrs->returnType = NULL;
  obj->funcAttrs->scope = createScope(obj, symtab->currentScope);
  return obj;
}
```

2.6 Hàm `createParameterObject`

Tạo đối tượng tham số, lưu hình thức truyền tham trị hoặc tham biến và con trỏ đến hàm hoặc thủ tục sở hữu tham số.

```
Object* createParameterObject(char *name, enum ParamKind kind,
                              Object* owner) {
  //TODO
  Object* obj = (Object*) malloc(sizeof(Object));
  strcpy(obj->name, name);
  obj->kind = OBJ_PARAMETER;
  obj->paramAttrs =
      (ParameterAttributes*) malloc(sizeof(ParameterAttributes));
  obj->paramAttrs->kind = kind;
  obj->paramAttrs->type = NULL;
  obj->paramAttrs->function = owner;
  return obj;
}
```

2.7 Hai điều chỉnh bổ sung

Sửa lỗi giải phóng kiểu mảng trong hàm `freeType()`:

Đoạn mã trước khi sửa:

```
case TP_ARRAY:
  freeType(type->elementType);
  freeType(type); // Dong gay loi
  break;
```

Dòng gây lỗi: `freeType(type);` gọi lại hàm với chính con trỏ kiểu mảng đang xử lý. Đối tượng này vẫn có lớp kiểu `TP_ARRAY`, nên hàm lại đi vào cùng nhánh và tiếp tục xử lý kiểu phần tử đã được giải phóng ở lần gọi trước. Điều này có thể gây truy cập bộ nhớ không hợp lệ hoặc giải phóng bộ nhớ nhiều lần.

Cách sửa: Thay `freeType(type)` bằng `free(type)`. Sau khi giải phóng kiểu phần tử, chương trình cần giải phóng trực tiếp vùng nhớ của chính đối tượng kiểu mảng.

Đoạn mã sau khi sửa:

```
case TP_ARRAY:
  freeType(type->elementType);
  // Giai phong truc tiep kieu mang,
  // tranh goi de quy lai chinh no.
  free(type);
  break;
```

Khởi tạo phạm vi trong hàm `initSymTab()`:

Lỗi ban đầu: Sau khi cấp phát bằng `malloc`, `currentScope` chưa được khởi tạo nhưng đã được sử dụng khi tạo phạm vi cho các hàm có sẵn. Giá trị chưa xác định này có thể được lưu vào con trỏ phạm vi ngoài của các hàm đó.

Cách sửa: Đặt `currentScope` về `NULL` ngay sau khi cấp phát bảng ký hiệu, trước khi tạo các hàm và thủ tục có sẵn.

```
symtab = (SymTab*) malloc(sizeof(SymTab));
// Chua co pham vi hien tai khi khoi tao cac ham co san.
symtab->currentScope = NULL;
symtab->globalObjectList = NULL;
```